Investigating the Factors Influencing Programmers' Acceptance of Machine-Generated Codes

Changwen Li 1360219

Word Count: 6217

Department of Computing and Information System
The University of Melbourne

September 17, 2023

Contents

1 Introduction **2**

2 Literature Review **3**
2.1 Code Generation System 3
2.2 Interaction between Programmers and Code Generation Systems 4
2.3 Factors Influencing Code Acceptance and Code Comprehension 5
2.3.1 Code Comprehension 5
2.3.2 Program Factors 6
2.3.3 Programmer Factors 7
2.3.4 Problem Factors 8

3 Research Questions **8**

4 Methodology **9**
4.1 Overview 9
4.2 Material 9
4.2.1 Programming Problem and Code Snippets 9
4.2.2 Questionnaires 11
4.3 Participants 11
4.4 Task 12
4.5 Dependent and Independent Variables 13
4.6 Data Analysis 13

5 Timeline **14**

6 Conclusion and Contribution **14**

1 Introduction

One of the most common hurdles in software engineering is turning ideas into programs [1]. A code generation system capable of automatically translating natural languages into codes could assist programmers, leading to a more efficient and productive programming paradigm [2, 3]. Automatically generating codes from high-level descriptions has been a historic challenge for decades in the computer science community [4]. The recent breakthrough in artificial intelligence, especially in large language models (LLMs), brings the computer science community closer to addressing this challenge [2,5]. There are multiple AI code generators that deliver impressive performance [5]. The AlphaCode, an AI code generator developed by DeepMind, achieved an average ranking in the top 54.3% among more than 5000 human programmers in coding contests on the Codeforces platform [2], showcasing the AI code generator's competitiveness in large coding contests. The Codex, a code generation model based on the large language model GPT-3 developed by OpenAI, demonstrated better performance than the average student on both introductory programming subjects [6] and advanced programming subjects [7]. In an introductory programming exam, Codex outperformed 76% of students [6]. In an advanced programming exam, Codex outscored 75% of students [7].

Despite the fact that recent advancements in artificial intelligence are reshaping software engineering [5, 8], very little research has investigated the interaction between human programmers and AI program generators [8, 9]. Some researchers conducted user experience studies of several code generators [3, 9–11]. They investigated the programmers' perception of code generators [3, 9] and found out that the programmers are willing to use these code generation tools although there are still challenges [3, 9]. The difficulties of using these code-generation tools are studied as well [3, 9–11]. One of the obstacles is that programmers are not aware of what kind of queries could be translated into codes [10]. Programmers also found it challenging to find and fix the subtle bugs in the generated codes [3, 9, 10]. Another challenge is prompt engineering, where users struggle to formulate natural language queries that enable the code generation system to yield performant outcomes [2, 11]. There are also papers investigating the performance and usage of code generation in computer science education [6, 7, 12–14]. These papers focus on how novice programmers interact with the code generation system [7, 13, 14]. The code generation system has both beneficial [14] and harmful [7, 13] influences on novice programmers. The code generation system could help to save time and alleviate pressure for novice programmers [14], but it could lead to over-reliance [7, 13].

Within the limited body of literature investigating the interaction between human programmers and AI code-generation systems, even less research focuses on understanding programmers' acceptance of codes generated by code-generation systems. It remains ambiguous under which conditions the programmers tend to accept and use the generated codes, and under which conditions the programmers tend to reject to use the generated codes. There is currently no research studying the specific factors that could influence programmers' acceptance of generated codes. In this research paper, we undertake a systematic classification of the potential factors that could influence programmers' willingness to accept the generated codes. We categorize the potential influential factors into three distinct groups: program factors, programmer factors and problem factors. The program factor refers to the factors inherent to the program itself, which represent the characteristics of the generated codes. Several examples of program factors include correctness, identifier naming style, comment quality, code complexity, indentation

and coding pattern. Programmer factors are tied to the attributes and conditions of the individual programmer, such as programming experience, language proficiency, domain knowledge familiarity and attitude towards AI coding assistants. Problem factors relate to the problem or task that the particular program is trying to solve, such as the scope of the problem and the difficulty of the problem. Details of the three types of factors will be further introduced in the literature review.

Research aims and objectives: This research aims to understand in which condition the programmers are willing to accept the machine. The objective of this research is to investigate which particular factors could influence programmers' acceptance of generated codes. There are multiple factors that could potentially influence programmers' acceptance. In consideration of feasibility, this research only selectively focuses on a subset of potential influential factors. For the program factors, we will investigate the influence of identifier naming style and comment quality. In terms of the programmer factors, we will assess the impact of programmers' programming experience, language proficiency, domain knowledge familiarity and attitudes towards AI. For the problem factor, our research focus will be on the perceived difficulty of the coding problem.

In section 2, we will review the corresponding literature. The related work regarding the code generation system, along with the interaction between human programmers and the code generation system will be introduced. The potential influential factors will then be introduced in detail. In section 3, the research question will be presented. In section 4, the methodology for this study will be introduced. We will explain the experiment design and the analysis methods. In section 5, we will showcase the timeline of this study. In section 6, we will make a conclusion for this research proposal and introduce the potential contributions.

2 Literature Review

2.1 Code Generation System

Code generation system refers to the system that can translate prompts in natural language into codes [2, 3]. The research of the code generation system has lasted for a long time [4, 15]. One of the earliest works can be traced back to the 1960s [9] when a program synthesizer could produce LISP codes from user-given instructions in the form of predicate calculus [15]. Although the necessity and feasibility of the automatic code generation system have been questioned in the early days [16], current academia and industry widely agree that being capable of translating natural language queries into codes automatically could be helpful in some scenarios [3]. For instance, the code generation system could help novice programmers achieve better performance in coding compared to those without access to the automatic code generation system [14]. With assistance from the code generation system, experienced programmers could save time by avoiding writing simple and repetitive codes [17, 18].

To automatically generate codes from natural language queries, there are mainly two classes of methods [3, 9]. The first one is based on search techniques. This method searches in some large code repositories and databases according to the users' intents or current programming contexts, and then it provides the existing codes based on the search results [3, 9]. Given the fact that there are several large code bases, such as GitHub and StackOverflow, it is possible that the codes needed by the programmer already exist on the code base [3]. The search method is utilized in a range of tasks, such as data handling in Microsoft Excel spreadsheet [19], bit-vector implementation in streaming program [20], and

the creation of regular expression [21]. However, a limitation of the code search method is that it only offers satisfactory performance to the domain-specific languages (DSL) whose syntax and language rules are less complicated than the general-purpose languages such as Java and C++ [9]. This limitation leads to unsatisfactory scalability if the program needs to be written in a general-purpose language.

The second method to translate natural language to codes relies on program generation based on machine learning, especially generative models [3,9]. This method is generally a more difficult but more effective and valuable approach [3]. Many code generation systems have integrated generative models [22–24]. The model RobustFill utilized the sequence-to-sequence (seq2seq) architecture based on the Long Short Term Memory (LSTM) model to perform code generation tasks [22]. Some researchers have investigated a learnable representation of code semantics, which enhances the performance of the code generators based on recurrent neural networks (RNN) [23]. Another group of researchers developed a transformer model that can understand the structure of the codes by integrating syntax tree [24].

The Large Language Model (LLM) makes a significant breakthrough in natural language processing (NLP) [25], and it has been utilized for code generation as well [8]. The Large Language Model (LLM) refers to the language model trained on a vast number of data and possesses a massive number of parameters [25]. An example of LLM is GPT-3, the third generation of Generative Pretrained Transformer released by OpenAI [6, 8, 25]. GPT-3 has 175 billion parameters, and the training data size is more than 570 GB [6]. GPT-3 is a general-purpose LLM that is not specifically trained for code generation [6]. OpenAI developed Codex, a language model which is specifically used for code generation. Codex is a descendant of GPT-3, with an additional 159 GB codes from more than 50 million GitHub repositories [6, 13]. Codex performs better than many previous code generation models, such as TabNine [7]. Codex has empowered a coding assistant tool, CoPilot, as an IDE plugin for end-users [13]. The newest version of GPT is GPT-4. GPT-4 outperforms GPT-3, with 75.6% correctness in solving easy LeetCode questions, compared to the correctness of 29.3% for GPT-3 [26].

2.2 Interaction between Programmers and Code Generation Systems

There are mainly two classes of methodologies adopted to understand the interaction between programmers and program generators. The first type of methodology utilizes a crowdsourcing data-driven framework to investigate data from social media posts [8]. Researchers discovered that the two most popular programming languages used on ChatGPT were Python and JavaScript [8]. Tasks related to code generation included bug fixing, assignment finishing and interview preparation [8]. The most common sentiments towards ChatGPT was fear, while happiness and anger were the least two common ones [8].

The second method is conducting a usability study of a code generation tool [3,5,9–11]. CoPilot, an IDE plug-in for code generation, was evaluated to understand the experience of using AI code generators [3, 5, 9]. By studying users' behaviour and thoughts while using CoPilot, researchers have identified two modes of interaction between programmers and program generators, which are acceleration mode and exploration mode [5]. In acceleration mode, programmers know what to do next and use CoPilot to speed up. In exploration mode, programmers are uncertain about the next step and use CoPilot to explore possible solutions. The productivity of CoPilot for end-users was investigated by another usability study [9]. The results illustrated that programmers preferred to use it in daily programming, although CoPilot did not necessarily decrease code completion time and increase code correctness rate [9]. This result indicates that the intrinsic promising performance in natural language processing facets

does not necessarily entail high productivity for end users [3].

Besides CoPilot, usability studies were also conducted on other code generation tools, such as GenLine [10]. The usability study showcased several difficulties faced by programmers when using code generation tools. Firstly, Programmers encounter difficulties in identifying and rectifying the subtle errors in the generated codes [10]. Secondly, programmers encounter difficulties in writing effective prompts. Programmers are unclear about what kind of natural language prompts can be translated to codes reliably by the code generation system. Programmers often struggle to direct the code generation system to generate desired codes [10]. The challenge of ineffective prompting indicates a gap between natural language and codes [11]. In order to bridge the gap between what the programmer intends and what the code generation system understands, the grounded abstraction matching approach is proposed [11]. The grounded abstraction matching suggests that a code generation system should be able to map users' naturalistic utterances into system actions, and then map the system actions back to naturalistic utterances. The users then can observe this grounded example and modify the natural language query to learn how the natural language prompt is mapped to the generated codes [11].

2.3 Factors Influencing Code Acceptance and Code Comprehension

The objective of this study is to explore the specific factors that influence programmers' acceptance of the codes produced by the code generation system. Although there is currently no literature investigating programmers' acceptance of machine-generated codes, the "computers are social actors" (CASA) [27] paradigm provides an insightful perspective. The CASA paradigm suggests that humans instinctively apply the same social heuristics used for human-to-human interactions to human-to-computer interactions [27]. Based on the CASA paradigm, it is plausible to extrapolate that the factors influencing programmers' acceptance of machine-generated codes can possibly overlap with the factors influencing their acceptance of human-crafted codes. Given the fact that code comprehension plays a pivotal role in software development and maintenance [28], code comprehension is highly related to code acceptance [29]. Therefore, the factors that influence code comprehension might potentially influence programmers' acceptance of machine-generated codes. In this research, we review the factors influencing code comprehension to shed light on the potential factors that could influence programmers' acceptance of machine-generated codes. We will also review other potential influential factors that are not studied by research about code comprehension.

2.3.1 Code Comprehension

Code comprehension is a crucial aspect of software engineering [29], which takes up 30% to 70% of a programmer's working time [30]. Many studies are dedicated to investigating which factors influence code comprehension [31–43]. The investigated factors include identifier names [31–36], comments [37–39], code structure [40,41], design pattern [42], indentation [43] and demographics [32,33,35,42,43]. The factors affecting code comprehension can be generally categorized into two classes, which are program factors and programmer factors. The program factors are the intrinsic factors of the program itself, such as identifier naming style and comment quality. The programmer factors represent the demographics of the programmer, including programming experience, language proficiency and domain knowledge familiarity.

2.3.2 Program Factors

The list of program factors includes identifier names [31–36], comments [37–39], code structure [40,41], design pattern [42] and indentation [43]. An identifier name is a token that distinguishes an entity, such as a function, class, or variable, within codes [31]. An early study conducted by Lawie et al. introduced the influence of identifier length and semantics [32]. It examined programmers' comprehension of several code snippets when the identifier names are written as full names (e.g. *isPrimeNumber*), abbreviations (e.g. *isPriNum*) and random letters (e.g. *a*). The result implies that the use of random single letters can significantly reduce both the accuracy and confidence of code comprehension [32]. Hofmeister et al. also focused on investigating the influence of full name identifiers, abbreviation identifiers, and random single letter identifiers on time needed for code comprehension [33]. Their findings underscored that the shorter identifier names typically need longer time to comprehend [33]. Avidan and Feitelson extended the research of identifier names by considering different types of identifiers [34]. They explored how different identifier naming styles affect code comprehension for function parameters and local variables. They discovered that the identifier naming style has a more notable influence on code comprehension when it is applied to function parameters [34]. Besides the research focusing on the length and descriptiveness of identifier names, a study undertaken by Binkley et al. focused on the superficial format of the identifier name, namely the camel case format (e.g. *variableName*) and underscore format (e.g. *variable_name*) [36]. Their findings highlight that camel case format is easier to comprehend regardless of the programmers' programming experience [36].

The significance of comments is emphasized in many coding guidelines because comments have a profound impact on code comprehension [44]. Several empirical studies have explored the influence of comments on code comprehension [37–39]. An early empirical study conducted by Takang et al. investigated whether the presence of the comments influences code comprehension. Its findings demonstrated that comments play a profound role in enhancing program comprehension, regardless of the identifier naming convention [37]. Nurvitadhi et al. expanded the study by examining how different types of comments influenced code comprehension [38]. Nurvitadhi et al. investigated how comments of the class and comments of the method influence Java code understanding. Method comments notably enhance comprehension, while class comments do not have a significant impact on code understanding [38]. Börstler and Paech further expanded the research by exploring how the quality of comments influences code comprehension [39]. They set up three variants for a code snippet, which were the snippet with good comments, the snippet with bad comments, and the snippet with no comments. They defined the "good comment" as the comment that provides additional information regarding the purpose of a certain code block, while the "bad comment" is the comment that only describes what the code block does. They found out that the perceived readability was affected by the existence of comments. However, in some cases, the code snippets with bad comments received unexpectedly high perceived readability. This finding suggests that the mere presence of comments could influence programmers' perception of the code's readability regardless of the comment quality. They discovered that there was no significant difference between participants' comprehension of code snippets with bad comments and code snippets with good comments [39].

Other program factors, including code structure [40, 41], design pattern [42], and indentation [43], were investigated as well. The code structure here refers to the arrangement, layout and organization of codes. Expression structures and looping manners are two examples of code structure. Cates et

al. delved into expression structures by investigating the effect of compound expressions and multiple simpler expressions [40]. In this context, the compound expression refers to an expression that combines multiple simpler expressions by operators, while using multiple simpler expressions usually introduces intermediate identifiers to the source codes. The findings highlighted that multiple simpler expressions with meaningful intermediate identifier names can reduce code comprehension time compared to compound expressions [40]. Johnson et al. studied the impact of looping manners on code comprehension [41]. They conducted an empirical study to scrutinize if minimizing loop depth and avoiding do-while loops can improve code readability. Their research results indicated that minimizing loop depth can enhance code comprehension while avoiding do-while loop has no statistically significant impact on code comprehension [41]. Abbes et al. conducted an empirical study to test if the design antipattern, including the Blob antipattern and Spaghetti antipattern, influences code comprehension [42]. The Blob antipattern refers to the design pattern where a dominant class takes up many roles, overshadowing other classes. The Spaghetti antipattern is characterized by tangled and complex control flow. The empirical study provided grounded evidence that the use of the Blob antipattern and the Spaghetti antipattern requires programmers to pay extra time and effort to understand the source codes [42]. The effect of code indentation is verified by Bauer et al. [43]. Their findings suggest that the use of indentation can improve code comprehension while the depth of indentation does not have a significant influence on code comprehension [43].

2.3.3 Programmer Factors

Programmer factors are the demographics of the programmer, including gender, intelligence, personality, programming experience, programming language proficiency and attitudes towards AI. Several empirical studies investigating the impacts of program factors on code comprehension also explore programmer factors by collecting demographic data and analyzing them [32, 33, 35, 42, 43]. For instance, Lawrie et al. explored the effect of programmer factors on code comprehension when they were studying the influence of identifier names [32]. They investigated the influence of gender, programming experience and programming language proficiency. They found out that male participants with more working experience were more confident in understanding source codes [32]. They also discovered that the identifier naming style has a more significant impact on programmers with lower programming proficiency [32]. There is also research solely focusing on investigating the impacts of programmer factors on code comprehension. Wagner and Wyrich studied the influence of intelligence and personality on code comprehension [45]. The results of their study showcased that there was a strong positive correlation between participants' general intelligence and code comprehension performance, and there was a weaker negative correlation between participants' conscientiousness and their performance on code comprehension [45].

Besides those programmer factors investigated in code comprehension research, such as programming experience and programming language proficiency, there are other programmer factors that have not been studied in code comprehension research. These factors include domain knowledge familiarity and attitudes towards artificial intelligence, and they could also potentially influence programmers' acceptance of machine-generated codes. Domain knowledge familiarity measures a programmer's familiarity with the background knowledge used in solving certain programming tasks [29]. For instance, in a programming task that needs to use a minimum heap, the domain knowledge familiarity is a programmer's familiarity with the knowledge regarding the minimum heap. Domain knowledge familiarity

is not studied in code comprehension research because researchers attempted to focus on the program factors, such as code structure and code semantics. Therefore, they ensure that the code snippets used in their research contain no domain knowledge, eliminating the influence of programmers' knowledge backgrounds [29]. However, domain knowledge familiarity could influence programmers' acceptance of machine-generated codes because it can influence programmers' interaction mode with code generators. As introduced before, there are two interaction modes between programmers and code generators [5]. When programmers are familiar with the domain knowledge, they will probably use the code generator to accelerate programming. When programmers are unfamiliar with the domain knowledge, they will probably use code generators to explore possible options. Based on the potential influence of domain knowledge familiarity on interaction modes, it is plausible to make the extrapolation that it could influence programmers' acceptance of machine-generated codes. Attitudes towards artificial intelligence could also potentially affect programmers' acceptance of machine-generated codes. If people trust certain technology, then they are more likely to accept and engage with this technology [46]. If programmers trust artificial intelligence, then they will probably be more willing to accept the codes generated by AI code generators.

2.3.4 Problem Factors

Problem factors refer to the attributes of the task or problem that the program needs to solve. The problem factors are not explored in the research regarding code comprehension. The reason is that participants are usually required to figure out what task the source codes are trying to solve [29], which means that the task or the problem of the codes is the consequence of code comprehension instead of the causal factors of code comprehension. Although problem factors are not the causal factors of code comprehension, they are potentially the causal factors of programmers' acceptance of machine-generated codes. When programmers use code generators such as Codex, they will need to describe the problem they want the code generators to address [11]. The influence of problem factors is worth investigating since the problem that the code solves is the starting point of interaction between programmers and code generators. An example of problem factors is the perceived difficulty of the programming task. The perceived difficulty of programming tasks could influence programmers' acceptance of machine-generated codes. A very complicated problem can overload a programmer's cognitive capacity [47]. When a programmer's cognitive capacity is overloaded, he might tend to use the codes generated by the code generator to alleviate cognitive burden.

3 Research Questions

The research question in our study is: *How do (A) programmer factors and (B) program factors influence programmers' acceptance of machine-generated codes under different levels of perceived problem difficulty?*

The programmer factors that will be studied in our research are *(A1) programming experience*, *(A2) programming language proficiency*, *(A3) domain knowledge familiarity* and *(A4) attitudes towards AI*. The program factors investigated in this study include *(B1) identifier naming style* and *(B2) comment quality*. The problem factor explored in this study is the *(C1) perceived problem difficulty*.

4 Methodology

4.1 Overview

We will address our research question by conducting controlled experiments based on online questionnaires. We will utilize an online survey to collect participants' demographics, including *programming experience*, *programming language proficiency*, *domain knowledge familiarity* and *attitudes towards AI*. We will provide a programming problem to the participants and ask them to write a code snippet to solve this programming problem on their own. After several minutes, the participants will rate the perceived difficulty of this programming problem. Then, a code snippet with certain types of *identifier naming style* and *comment quality* will be provided to the participants. Participants will be told that the code snippets provided to them are generated by an AI code generation system. The participants will need to submit a code snippet to solve this programming task. They can use the entire provided code snippet or integrate parts of the provided code snippet into their submitted codes. After participants finish their questionnaires, we will compare their submitted codes and the provided codes to measure to what extent the participants accept the machine-generated codes.

4.2 Material

4.2.1 Programming Problem and Code Snippets

The programming problem we chose is an algorithm problem from LeetCode, an online platform with an adequate number of programming problems [48]. The chosen programming problem [1] requires the programmer to write a data structure to simulate the Least Recent Used (LRU) caching mechanism with *O(1)* for *get* and *set* operations [49]. To simulate the LRU mechanism in low time complexity *O(1)*, the Hash map and double-linked list are needed [2]. We will modify the programming problem to fit our research. We will delete the introductions of LRU in the original programming problem so that we can examine how familiarity with domain knowledge of LRU influences the acceptance of codes generated by code generators. This programming problem is chosen because of the following reasons. First, it contains domain knowledge of LRU, Hash map and double-linked list, which enables us to test the correlation between domain knowledge familiarity and acceptance of machine-generated codes. Second, this problem represents a common real-life scenario of using a code generator. Students studying computer science use code generators to help them with their algorithm and data structure assignments [13], and practitioners in the IT industry use code generators to review algorithms for interview and coding test preparations [8].

The code snippets are modified with certain types of identifier naming styles and comment quality to examine the influence of program factors on the acceptance of machine-generated codes. Regarding *(B1) identifier naming style*, we mirrored the code snippet variation design of the study conducted by Schankin et al. [35] and set up the following variants:

- *B11. Descriptive Compound Identifier*: The descriptive compound identifier is an identifier consisting of several words, which introduce the nature of the identifier in a descriptive way.

[1] Original website of the chosen programming problem: https://leetcode.com/problems/lru-cache/
[2] Website of the solutions to the chosen programming problem: https://leetcode.com/problems/lru-cache/solutions/

- *B12. Short Identifier*: The short identifier is usually a single-word identifier with less descriptiveness.

The examples of *B11. Descriptive Compound Identifier* variants and *B12. Short Identifier* variants are shown in the Table 1 below.

Table 1: Examples of identifier naming styles

Descriptive compound identifier	Short identifier
cacheMapKeyToAddress	map
cacheRecordList	list
keyToErase	key
valueToUpdate	value

In terms of comment quality, we adopted the same variants as those presented in Borstler and Paech's study [39]. The variants of comment quality we will adopt are:

- *B21. Good Comment*: The comments in the code snippet are all useful and meaningful comments with additional information that introduces the purposes of the code block. An example of a good comment is shown in listing 1.

```
1        // Build a Hash table to retrieve the targeted LRU record. This table
             maps the key of the LRU record to the address of the record.
2        unordered_map<int, LRURecordNode*> cacheMap;
```

Listing 1: An example of good comment

- *B22. Bad Comment*: The comments in the code snippet only repeat what the code does, without additional information introducing the purposes of the code block. An example of a bad comment is shown in listing 2.

```
1        // Build a Hash table that maps a number to an address of
             LRURecordNode.
2        unordered_map<int, c*> cacheMap;
```

Listing 2: An example of bad comment

- *B23. No Comment*: There is no comment in the code block.

There are six combinations generated from the two variants of factor *(B1) identifier naming style* and the three variants of factor *(B2) comment quality*. Therefore, we will provide six versions of code snippets that combine all variants of identifier naming style and comment quality. A detailed description of the six code snippet versions is demonstrated in table 2.

The code snippets will be written in C++. We plan to input the problem description along with our specifications regarding identifier naming style and comment quality into ChatGPT. After that, we will obtain the generated codes. In order to ensure that all the code snippets align with the specifications of identifier naming style and comment quality, we will adopt the same approach utilized by Truede and Robillard in their research regarding StackOverflow code fragments [50]. We will invite three programmers with at least five years of programming experience to review if the generated codes align with the specifications. They will collectively vote to determine if the code aligns with the provided

Table 2: Code snippet versions and descriptions

Code Snippet Version	Combined Variants	Description
V11	B11 & B21	Descriptive Compound Identifier & Good Comment
V12	B11 & B22	Descriptive Compound Identifier & Bad Comment
V13	B11 & B23	Descriptive Compound Identifier & No Comment
V21	B12 & B21	Short Identifier & Good Comment
V22	B12 & B22	Short Identifier & Bad Comment
V23	B12 & B23	Short Identifier & No Comment

specifications. If any one of them disagrees that the specifications are followed, then they will discuss and revise the code snippets until all evaluators reach an agreement. We will publish the code snippets on GitHub so that future researchers can use our code snippets to reproduce our research or conduct new research.

4.2.2 Questionnaires

This study will utilize Qualtrics questionnaires to obtain raw data. Qualtrics is a popular survey platform that is used by many research with participant recruitment [51]. We will capture raw data on the factor *(A1) programming experience* by requesting participants to specify their years of experience in programming. The raw data of factor *(A2) programming language familiarity*, *(A3) domain knowledge familiarity* and *(A4) attitudes towards AI* will be obtained by employing eleven-point Likert Scales. To assess the participants' attitudes towards AI, we used the Attitudes Towards Artificial Intelligence (ATAI) scale proposed by Sindermann et al [52]. The ATAI scale consists of five eleven-point Likert scale questions with two of them evaluating acceptance of AI and three of them evaluating fear of AI. Participants' fear of AI is calculated by averaging participants' responses to the two questions regarding fear of AI. Participants' acceptance of AI is calculated by averaging participants' responses to the three questions regarding acceptance of AI. The overall attitudes towards AI are then calculated by averaging the fear of AI and the acceptance of AI. We employ the ATAI scale due to its simplicity and effectiveness. The ATAI scale consists of only five simple questions. Its simplicity guarantees that it will not take the participants a lot of time to finish them. The ATAI scale is also effective. The efficacy of the ATAI scale has been empirically validated and found to be commendable [52].

The questionnaire is also the place where the participants see the provided code snippets and write their own code snippets. To mimic the real-life programming experience, we will showcase the provided code snippets with syntax highlights and enable participants to write their codes in a code editing box. We will achieve syntax highlights and code editors by embedding HTML, CSS and JavaScript programs in the questionnaire.

4.3 Participants

The minimum number of participants needed in our research is 180. As mentioned in the table 2 , there are six versions of code snippets. For each code version, we plan to invite at least 30 participants to reduce sampling bias and increase generalizability. Therefore, we plan to recruit at least 180 participants. To further reduce bias and enhance generalizability, we will recruit participants with diverse backgrounds. We will recruit participants with different levels of programming experience. We will

recruit undergraduate computer science students, graduate computer science students and programmers working in the IT industry.

4.4 Task

Each participant will be randomly assigned a questionnaire with one of the six code versions. The task for the participants is to finish the questionnaire. The task can be split into four phases:

The first phase is the ethical briefing and study introduction phase. The participant will start the online survey by signing an ethical consent. The ethical consent will inform the participants that this study has been approved by the University of Melbourne, and their responses will be anonymous. After signing the ethical consent, we will briefly introduce the purpose and procedures of this online questionnaire.

The second phase is the demographic data collection phase. Participants will fill out the questions in the survey, providing information on some programmer factors, including *(A1) programming experience*, *(A2) programming language proficiency* and *(A4) attitudes towards AI*. However, the information on programmer factor *(A3) domain knowledge familiarity* will not be collected in the second phase. The question regarding *(A3) domain knowledge familiarity* will ask participants how familiar they are with the domain knowledge. In this research, we will ask the participants how familiar they are with the LRU cache mechanism, the Hash map, and the double-linked list. Asking this question might give the participants a hint that the programming problem needs to use a Hash map and double-linked list, which could unintentionally decrease the perceived difficulty of the programming problem and make it easier for the participants to understand the provided snippets. Therefore, we will not collect the information of *(A3) domain knowledge familiarity* until the participants submit their codes.

The third phase is the self-coding phase. We will show them the programming problems and require them to finish the programming problem on their own. The participants will be notified that the compensation they receive depends on the quality of the codes written by themselves. We will tell them that they will only receive the minimum amount of compensation if they produce low-quality codes independently. In this way, the participants will have stronger motivation to finish the programming problem well. Nevertheless, we will give every participant the highest amount of compensation to avoid potential arguments about the quality of their codes.

The fourth phase is the code integration phase. In the fourth phase, we will provide a code snippet and notify the participants that this is a code snippet generated by a code generation system. Before showing the provided codes, we will ask the participants to give a rating on the difficulty of the programming problem. We ask the question of the difficulty before providing the code snippets to reduce participants' bias. If we ask the question regarding the difficulty after showing them the provided code snippets, they might underestimate the difficulty of the problem because they have already seen the answer proposed by the provided code snippet. After showing them the provided code snippet, they will be notified that they have the option to use the entire provided code, integrate part of the provided code into their original code, or choose not to use the provided code at all. The participants will need to submit their codes after the time limit of the fourth phase expires. We adopt a similar approach used in the third phase to encourage participants' motivation to finish the task with more effort. We will tell them that the compensation they receive also depends on the quality of their final submitted codes. After the participants submit their codes, we will ask them to rate their *(A3) domain knowledge familiarity*.

4.5 Dependent and Independent Variables

The independent variables of this research are the *(A) programmer factors*, *(B) program factors* and *perceived problem difficulty*. The *(A) programmer factors* and *perceived problem difficulty* will be measured by the values obtained from the online survey, and the *(B) program factors* will be characterized by the version of codes with different variants of *(B1) identifier naming style* and *(B2) comment quality*.

The dependent variable is the acceptance of the machine-generated codes. To measure the acceptance of machine-generated codes, we will use the code similarity between the provided code snippets and the participants' submitted code snippets. The code similarity will be calculated via MOSS (Measure Of Software Similarity), a popular automatic code similarity checking system [53].

4.6 Data Analysis

The dependent variable, programmers' acceptance of machine-generated codes, is numerical data. The data of variables *A1*, *A2*, *A3*, *A4* and *perceived problem difficulty* will be obtained via 11-point Likert scales ranging from 0-10. In this study, we treat variables *A1-A4* and *perceived problem difficulty* as numerical data for two primary reasons: Firstly, treating them as numbers facilitates parametric statistical analysis [54]. Secondly, a Likert scale with 11 points can be approximated as continuous data, simulating interval scale [55]. The variables *B1* and *B2* are categorical data.

To investigate the influence of some variables on code acceptance, one potential method is to conduct statistical significance testing of each individual variable. In code comprehension research, many researchers conduct significance testing, such as ANOVA [33,42], t-test [41,42], chi-square test [40,50], and Whitney-Mann test [34,40,42], on individual variables. However, this approach is not suitable for my research. The reason is there are a lot of variables investigated in my research. The influence of these variables tangle with each other [43]. For instance, a participant with a lot of programming experience may tend to accept the generated codes when he is not familiar with the domain knowledge. However, he might not accept the generated codes when he is familiar with the domain knowledge because he can finish the codes on his own. In this case, there might be no significant influence of programming experience on code acceptance. This phenomenon is the "interaction effect", where the effect of one independent variable (e.g., programming experience) on the dependent variable (code acceptance) depends on the level of another independent variable (domain knowledge familiarity) [56].

To mitigate the interaction effect and get more insight into the possible influential factors, we plan to use the generalized linear model. This model is used by some code comprehension studies [32,57] as well. The model looks like this:

$$Acceptance = \beta_0 + \beta_1(A_1) + \beta_2(A_2) + \beta_3(A_3) + \beta_4(A_4) + \beta_5(B_1) + \beta_6(B_1) + \beta_7(C_1) + \beta_{13}(A_1A_3)...$$

In this model, we add some interaction terms to evaluate the interaction effect [32]. The interaction term $\beta_{13}(A_1A_3)$ assesses the interaction effect between A1 and A3. We will also add other interaction terms to evaluate the interaction effect between more variables. We will add additional dummy variables for B1 and B2 as they are categorical variables. T-tests are used to test the significance of coefficients in a regression model [58]. After putting the raw data to fit this model, we will conduct t-tests to examine the significance of each coefficient in our model. A p-value smaller than 0.05 suggests that the coefficient is a significant predictor, indicating a statistically significant relationship between the coefficient and the

dependent variable [59].

5 Timeline

Tasks	Semester 2, 2023												Semester 1, 2024											
	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3	4	5	6	7	8	9	10	11	12
Phase I: Preliminary Study																								
Identify the research topic.																								
Literature review.																								
Identify the research gap and research questions.																								
Write a research proposal.																								
Phase II: Piloting and Experiment Design																								
Design experiment questionnaires.																								
Pilot test and presentation preparation.																								
Revise the experiment design.																								
Recruit participants.																								
Conduct experiments and send questionnaires.																								
Phase III: Data Analysis																								
Confirm suitable statistical analysis methods																								
Conduct data analysis.																								
Write Thesis.																								

6 Conclusion and Contribution

This research aims to investigate the factors that influence programmers' acceptance of machine-generated codes. The investigated factors are classified into three categories, which are programmer factors, program factors, and problem factors. We plan to conduct a controlled experiment to investigate the influence of different identifier naming styles and comment qualities. After obtaining the raw data, we will use the generalized linear model and t-tests for data analysis.

Our research will make three contributions. Firstly, it addresses the research gap. Current research regarding code generators mainly focuses on the technical aspects, and very few studies explore the usability of code generators [8, 9]. Within the limited body of usability studies on code generators, none of them study the factors influencing programmers' acceptance of machine-generated codes. Our research will be the first study that assesses which specific factors influence programmers' acceptance of codes generated by code generators, which addresses the research gap. Moreover, this research provides insight into the interaction between programmers and program generators. We will uncover the factors that could influence programmers' acceptance of machine-generated codes. Furthermore, our research provides guidelines for code generators to enhance usability. Future code generators can utilize our research findings to produce codes that are more likely to be accepted by programmers. Our research will contribute to a harmonious integration of code generation systems into the software engineering ecosystems.

References

[1] P. Mohagheghi and R. Conradi, "Quality, productivity and economic benefits of software reuse: a review of industrial studies," *Empirical Software Engineering*, vol. 12, pp. 471–516, 2007.

[2] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. S. Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[3] F. F. Xu, B. Vasilescu, and G. Neubig, "In-ide code generation from natural language: Promise and challenges," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, mar 2022.

[4] Z. Manna and R. J. Waldinger, "Toward automatic program synthesis," *Commun. ACM*, vol. 14, p. 151–165, mar 1971.

[5] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," *Proc. ACM Program. Lang.*, vol. 7, apr 2023.

[6] J. Finnie-Ansley, P. Denny, B. A. Becker, A. Luxton-Reilly, and J. Prather, "The robots are coming: Exploring the implications of openai codex on introductory programming," in *Proceedings of the 24th Australasian Computing Education Conference*, ACE '22, (New York, NY, USA), p. 10–19, Association for Computing Machinery, 2022.

[7] J. Finnie-Ansley, P. Denny, A. Luxton-Reilly, E. A. Santos, J. Prather, and B. A. Becker, "My ai wants to know if this will be on the exam: Testing openai's codex on cs2 programming exercises," in *Proceedings of the 25th Australasian Computing Education Conference*, ACE '23, (New York, NY, USA), p. 97–104, Association for Computing Machinery, 2023.

[8] Y. Feng, S. Vanam, M. Cherukupally, W. Zheng, M. Qiu, and H. Chen, "Investigating code generation performance of chat-gpt with crowdsourcing social data," in *Proceedings of the 47th IEEE Computer Software and Applications Conference*, pp. 1–10, 2023.

[9] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*, CHI EA '22, (New York, NY, USA), Association for Computing Machinery, 2022.

[10] E. Jiang, E. Toh, A. Molina, K. Olson, C. Kayacik, A. Donsbach, C. J. Cai, and M. Terry, "Discovering the syntax and strategies of natural language programming with generative language models," in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, CHI '22, (New York, NY, USA), Association for Computing Machinery, 2022.

[11] M. X. Liu, A. Sarkar, C. Negreanu, B. Zorn, J. Williams, N. Toronto, and A. D. Gordon, ""what it wants me to say": Bridging the abstraction gap between end-user programmers and code-generating large language models," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, (New York, NY, USA), Association for Computing Machinery, 2023.

[12] S. Sarsa, P. Denny, A. Hellas, and J. Leinonen, "Automatic generation of programming exercises and code explanations using large language models," in *Proceedings of the 2022 ACM Conference*

on International Computing Education Research - Volume 1, ICER '22, (New York, NY, USA), p. 27–43, Association for Computing Machinery, 2022.

[13] B. A. Becker, P. Denny, J. Finnie-Ansley, A. Luxton-Reilly, J. Prather, and E. A. Santos, "Programming is hard - or at least it used to be: Educational opportunities and challenges of ai code generation," in *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, SIGCSE 2023, (New York, NY, USA), p. 500–506, Association for Computing Machinery, 2023.

[14] M. Kazemitabaar, J. Chow, C. K. T. Ma, B. J. Ericson, D. Weintrop, and T. Grossman, "Studying the effect of ai code generators on supporting novice learners in introductory programming," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, (New York, NY, USA), Association for Computing Machinery, 2023.

[15] R. J. Waldinger and R. C. T. Lee, "Prow: A step toward automatic program writing," in *Proceedings of the 1st International Joint Conference on Artificial Intelligence*, IJCAI'69, (San Francisco, CA, USA), p. 241–252, Morgan Kaufmann Publishers Inc., 1969.

[16] E. W. Dijkstra, "On the foolishness of "natural language programming"," *Program Construction: International Summer School*, pp. 51–53, 2005.

[17] D. Nam, A. Horvath, A. Macvean, B. Myers, and B. Vasilescu, "Marble: Mining for boilerplate code to identify api usability problems," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 615–627, 2019.

[18] V. Raychev, M. Vechev, and E. Yahav, "Code completion with statistical language models," *SIGPLAN Not.*, vol. 49, p. 419–428, jun 2014.

[19] S. Gulwani, "Automating string processing in spreadsheets using input-output examples," *SIGPLAN Not.*, vol. 46, p. 317–330, jan 2011.

[20] A. Solar-Lezama, R. Rabbah, R. Bodík, and K. Ebcioğlu, "Programming by sketching for bit-streaming programs," in *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, (New York, NY, USA), p. 281–294, Association for Computing Machinery, 2005.

[21] T. Zhang, L. Lowmanstone, X. Wang, and E. L. Glassman, "Interactive program synthesis by augmented examples," in *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*, UIST '20, (New York, NY, USA), p. 627–648, Association for Computing Machinery, 2020.

[22] J. Devlin, J. Uesato, S. Bhupatiraju, R. Singh, A. rahman Mohamed, and P. Kohli, "RobustFill: Neural program learning under noisy I/O," in *Proceedings of the 34th International Conference on Machine Learning* (D. Precup and Y. W. Teh, eds.), vol. 70 of *Proceedings of Machine Learning Research*, pp. 990–998, PMLR, 06–11 Aug 2017.

[23] T. Ben-Nun, A. S. Jakobovits, and T. Hoefler, "Neural code comprehension: A learnable representation of code semantics," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[24] S. Kim, J. Zhao, Y. Tian, and S. Chandra, "Code prediction by feeding trees to transformers," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 150–162, 2021.

[25] E. Kasneci, K. Sessler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günnemann, E. Hüllermeier, S. Krusche, G. Kutyniok, T. Michaeli, C. Nerdel, J. Pfeffer, O. Poquet, M. Sailer, A. Schmidt, T. Seidel, M. Stadler, J. Weller, J. Kuhn, and G. Kasneci, "Chatgpt for good? on opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, p. 102274, 2023.

[26] R. OpenAI, "Gpt-4 technical report," *arXiv*, pp. 2303–08774, 2023.

[27] C. Nass and Y. Moon, "Machines and mindlessness: Social responses to computers," *Journal of social issues*, vol. 56, no. 1, pp. 81–103, 2000.

[28] G. Broccia, A. Ferrari, M. Ter Beek, W. Cazzola, L. Favalli, and F. Bertolotti, "Evaluating a language workbench: from working memory capacity to comprehension to acceptance," in *2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC)*, pp. 54–58, 2023.

[29] D. G. Feitelson, "Considerations and pitfalls in controlled experiments on code comprehension," in *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*, pp. 106–117, 2021.

[30] X. Xia, L. Bao, D. Lo, Z. Xing, A. E. Hassan, and S. Li, "Measuring program comprehension: A large-scale field study with professionals," *IEEE Transactions on Software Engineering*, vol. 44, no. 10, pp. 951–976, 2018.

[31] A. Peruma and C. D. Newman, "Understanding digits in identifier names: An exploratory study," in *Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering*, NLBSE '22, (New York, NY, USA), p. 9–16, Association for Computing Machinery, 2023.

[32] D. Lawrie, C. Morrell, H. Feild, and D. Binkley, "What's in a name? a study of identifiers," in *14th IEEE International Conference on Program Comprehension (ICPC'06)*, pp. 3–12, 2006.

[33] J. Hofmeister, J. Siegmund, and D. V. Holt, "Shorter identifier names take longer to comprehend," in *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 217–227, 2017.

[34] E. Avidan and D. G. Feitelson, "Effects of variable names on comprehension: An empirical study," in *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*, pp. 55–65, 2017.

[35] A. Schankin, A. Berger, D. V. Holt, J. C. Hofmeister, T. Riedel, and M. Beigl, "Descriptive compound identifier names improve source code comprehension," in *Proceedings of the 26th Conference on Program Comprehension*, ICPC '18, (New York, NY, USA), p. 31–40, Association for Computing Machinery, 2018.

[36] D. Binkley, M. Davis, D. Lawrie, and C. Morrell, "To camelcase or under_score," in *2009 IEEE 17th International Conference on Program Comprehension*, pp. 158–167, 2009.

[37] A. A. Takang, P. A. Grubb, R. D. Macredie, *et al.*, "The effects of comments and identifier names on program comprehensibility: an experimental investigation," *J. Prog. Lang.*, vol. 4, no. 3, pp. 143–167, 1996.

[38] E. Nurvitadhi, W. W. Leung, and C. Cook, "Do class comments aid java program understanding?," in *33rd Annual Frontiers in Education, 2003. FIE 2003.*, vol. 1, pp. T3C–T3C, 2003.

[39] J. Börstler and B. Paech, "The role of method chains and comments in software readability and comprehension—an experiment," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 886–898, 2016.

[40] R. Cates, N. Yunik, and D. G. Feitelson, "Does code structure affect comprehension? on using and naming intermediate variables," in *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*, pp. 118–126, 2021.

[41] J. Johnson, S. Lubo, N. Yedla, J. Aponte, and B. Sharif, "An empirical study assessing source code readability in comprehension," in *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 513–523, 2019.

[42] M. Abbes, F. Khomh, Y.-G. Guéhéneuc, and G. Antoniol, "An empirical study of the impact of two antipatterns, blob and spaghetti code, on program comprehension," in *2011 15th European Conference on Software Maintenance and Reengineering*, pp. 181–190, 2011.

[43] J. Bauer, J. Siegmund, N. Peitek, J. C. Hofmeister, and S. Apel, "Indentation: Simply a matter of style or support for program comprehension?," in *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*, pp. 154–164, 2019.

[44] R. C. Martin, *Clean code: a handbook of agile software craftsmanship*. Pearson Education, 2009.

[45] S. Wagner and M. Wyrich, "Code comprehension confounders: A study of intelligence and personality," *IEEE Transactions on Software Engineering*, vol. 48, no. 12, pp. 4789–4801, 2022.

[46] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Human factors*, vol. 46, no. 1, pp. 50–80, 2004.

[47] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive science*, vol. 12, no. 2, pp. 257–285, 1988.

[48] N. Nguyen and S. Nadi, "An empirical evaluation of github copilot's code suggestions," in *Proceedings of the 19th International Conference on Mining Software Repositories*, MSR '22, (New York, NY, USA), p. 1–5, Association for Computing Machinery, 2022.

[49] A. Dan and D. Towsley, "An approximate analysis of the lru and fifo buffer replacement schemes," in *Proceedings of the 1990 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*, SIGMETRICS '90, (New York, NY, USA), p. 143–152, Association for Computing Machinery, 1990.

[50] C. Treude and M. P. Robillard, "Understanding stack overflow code fragments," in *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 509–513, 2017.

[51] T. C. Boas, D. P. Christenson, and D. M. Glick, "Recruiting large online samples in the united states and india: Facebook, mechanical turk, and qualtrics," *Political Science Research and Methods*, vol. 8, no. 2, pp. 232–250, 2020.

[52] C. Sindermann, P. Sha, M. Zhou, J. Wernicke, H. S. Schmitt, M. Li, R. Sariyska, M. Stavrou, B. Becker, and C. Montag, "Assessing the attitude towards artificial intelligence: Introduction of a short measure in german, chinese, and english language," *KI-Künstliche intelligenz*, vol. 35, pp. 109–118, 2021.

[53] T. Lancaster and F. Culwin, "A comparison of source code plagiarism detection engines," *Computer Science Education*, vol. 14, no. 2, pp. 101–112, 2004.

[54] I. E. Allen and C. A. Seaman, "Likert scales and data analyses," *Quality progress*, vol. 40, no. 7, pp. 64–65, 2007.

[55] H. Wu and S.-O. Leung, "Can likert scales be treated as interval scales?—a simulation study," *Journal of social service research*, vol. 43, no. 4, pp. 527–532, 2017.

[56] J. Jaccard and R. Turrisi, *Interaction effects in multiple regression*. No. 72, Sage, 2003.

[57] S. Kolodziej, "Empirical assessment of software documentation strategies: A randomized controlled trial," in *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*, pp. 1296–1295, 2019.

[58] S. Liu, M. Lu, H. Li, and Y. Zuo, "Prediction of gene expression patterns with generalized linear regression model," *Frontiers in Genetics*, vol. 10, p. 120, 2019.

[59] T. K. Kim, "T test as a parametric statistic," *Korean journal of anesthesiology*, vol. 68, no. 6, pp. 540–546, 2015.